

CSL302: Compiler Design

Code Generation and Optimizations

Vishwesh Jatala

Assistant Professor

Department of CSE

Indian Institute of Technology Bhilai

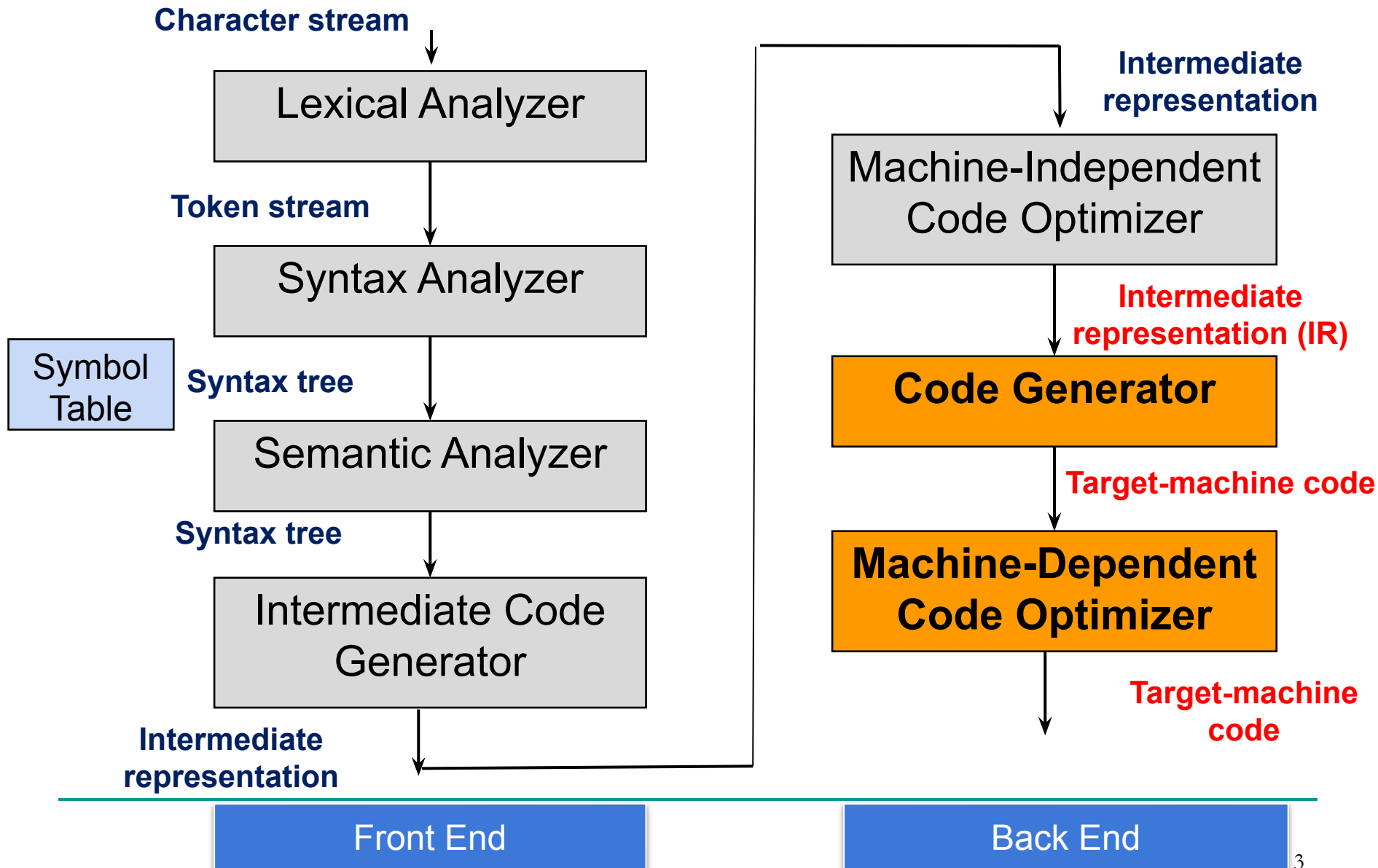
vishwesh@iitbhilai.ac.in



Acknowledgement

- References for today's slides
 - ▣ *Prof. Y. N Srikant, IISc Bangalore*
 - *<https://nptel.ac.in/content/storage2/courses/106108052/module4/code-gen-part-3.pdf>*
 - ▣ *Course textbook*

Compiler Design



Outline

- Introduction to code generation
- Challenges in code generation
- Code generation algorithm
 - Example
- Machine dependent optimizations
 - Peephole optimizations

Code Generation

- *Goal: Map IR program into a code that can be executed on a target machine*
- **Input:** Symbol table, IR (three-address code (TAC))
- **Output:** Target machine code
- **Assumptions:**
 - ❑ No syntactic and semantic errors
 - ❑ Type checking has been done

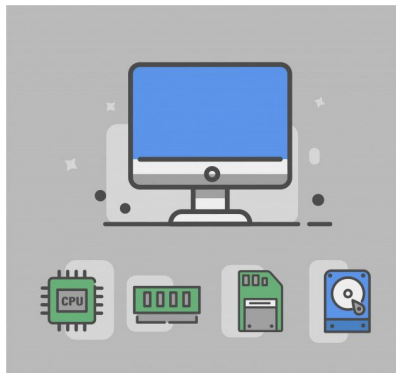
Desired Characteristics



Correct: Preserve semantics



Fast: Less execution time



Resource efficient



Energy efficient

Reality

- Generating the optimal code is undecidable
- Several sub-problems are NP-Complete
 - Register allocation
 - Evaluation order
- Practical approaches use heuristics

Challenges in Code Generation

- Target architecture
- Instruction selection
- Register allocation
- Evaluation order

Challenges: Target Architecture

- Instruction set architecture has impact on difficulty of code generation
 - Common architectures: RISC and CISC

RISC

Many Registers

Simple-addressing modes and ISA

CISC

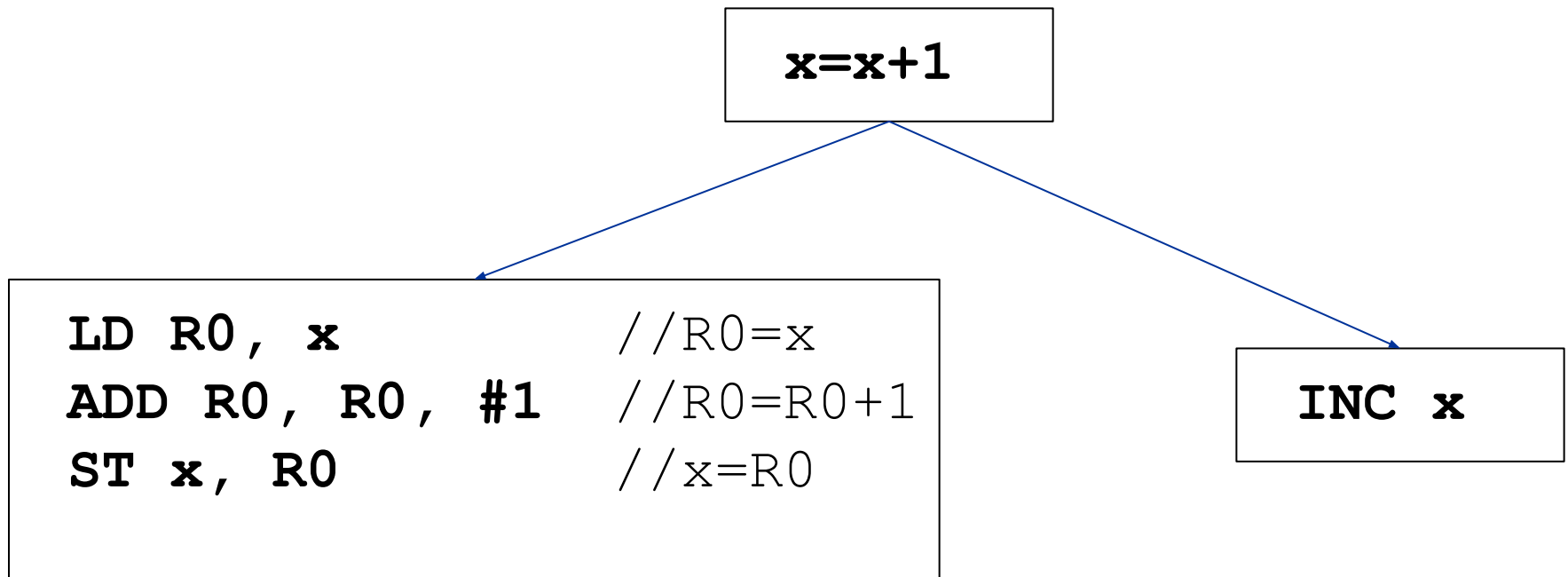
Few Registers

Variety of addressing modes, variable length instructions

We will use RISC in the lecture

Challenges: Instruction Selection

- Instruction selection is critical for performance



Challenges: Instruction Selection

- Naive implementation can lead to redundant loads/stores

x=y+z



```
LD R0, y
ADD R0, R0, z
ST x, R0
```

a=b+c
d=a+e



```
LD R0, b
ADD R0, R0, c
ST a, R0
LD R0, a
ADD R0, R0, e
ST d, R0
```

Challenges: Register Allocation

- Registers are fastest storage units
- Registers are few
- Efficient register utilization is critical

a	:=	c	+	d
e	:=	a	+	b
f	:=	e	-	1

a, **e**, and **f** can be allocated
in 1 register!

Optimal register allocation is NP-Complete

Challenges: Evaluation Order

- Order in which computations are performed can affect of performance
- Some computation orders require few registers
- Picking the best order is NP-Complete

Target Language

- Assume a simple target assembly code
- Instructions:
 - ❑ Load: *LD dst, addr*
 - ❑ Store: *ST x, reg*
 - ❑ Computation operations: *OP dst, src1, src2*
 - OP can be ADD, SUB, MUL, etc.
 - ❑ Unconditional jumps: *BR L*
 - ❑ Conditional jumps: *BCond reg, L*

Exercise-1

- **Manually** generate the code for the following TAC according to the defined target architecture
 - ❑ Assume there is no limit on the number of registers.

$x = y + z$

Exercise

- **Manually** generate the code for the following TAC according to the defined target architecture
 - ❑ Assume there is no limit on the number of registers.

$x = y + z$



```
LD R1, y
LD R2, z
ADD R1, R1, R2
ST x, R1
```


Exercise-2

- **Manually** generate the code for the following TAC according to the defined target architecture
 - ❑ Assume there is no limit on the number of registers.

t = a-b

u = a-c

v = t+u

d = v+u